

Push-Benachrichtigungen

Was steht, was fehlt und wie du es besorgst · Stand 4. Oktober 2026

So liest du das: **Rot** musst du selbst machen — es hängt an deinen Konten. **Grün** ist erledigt. **Gelb** heißt: angefangen oder von hier aus nicht nachprüfbar. **Der Server-Teil ist seit dem 24.09.2026 fertig und läuft** — offen ist nur noch, was Apple-Konten und Stores betrifft.

Die Übersicht

#	TEIL	STATUS	WER
1	Bezahlte Apple-Mitgliedschaft im Team <code>HACJC6623Z</code>	wohl erledigt	du
2	Push-Berechtigung für die App-ID <code>app.matchup.mobile</code>	wohl erledigt	du
3	Key ID des APNs-Schlüssels — <code>9JF35BFS9Q</code>	da	—
4	Schlüsseldatei <code>AuthKey_9JF35BFS9Q.p8</code> in <code>~/keys/</code>	da	—
5	Team ID — <code>HACJC6623Z</code>	da	—
6	Bundle ID — <code>app.matchup.mobile</code>	da	—
7	Firebase-Projekt <code>matchup-f9e83</code> , Analytics aus	da	—
8	<code>GoogleService-Info.plist</code> + <code>google-services.json</code>	im Projekt	—
9	Dienstkonto-Schlüssel als Secret <code>FIREBASE_DIENSTKONT0</code>	gesetzt	—
10	APNs-Schlüssel bei Firebase hinterlegt	greift nicht	du
11	Datenschutzerklärung veröffentlicht · Store-Angaben	Seite live	du
12	<code>aps-environment</code> , Berechtigungsabfrage, Geräte-Tokens	fertig	
13	Tabellen, Auslöser und Einstellungen je Nutzer	Migration 0129	
14	Edge Function <code>push</code> — ausgespielt, drei Cron-Jobs aktiv	läuft	

Nachgemessen, nicht behauptet. 24.09.: Die Tabellen `push_geraete`, `push_einstellungen` und `push_auftraege` stehen, die Function antwortet `{"auftraege":0,"gesendet":0}`, der Minutenlauf kommt mit `200` am Gateway vorbei. 02.10.: Das Dienstkonto holt einen OAuth-Token. 04.10.: In `push_geraete` liegt seit dem 25.09. ein **iOS-Token** — der Testpush dorthin scheitert aber an Apple (Schritt 6). **Zu Punkt 1 und 2:** Das Zertifikat auf diesem Mac trägt `OU=HACJC6623Z`, also die richtige Team-ID, und es gab laut Projekt schon einen TestFlight-Build — den gibt es nur mit bezahltem Programm. Dass iOS überhaupt einen Push-Token ausgestellt hat, setzt zudem `aps-environment` voraus. Beides bleibt dennoch unbewiesen, bis der Testpush durchgeht. Ein *Distribution*-Zertifikat fehlt weiterhin — das brauchst du aber erst für TestFlight und den Store, nicht zum Testen.

Vorher lesen — drei Regeln

- Die **Key ID** (`9JF35BFS9Q`) ist keine Geheimnummer, die darfst du weitergeben.

- Die **Datei .p8** ist das **Geheimnis**. Ihren Inhalt nirgends hineinkopieren — auch nicht in einen Chat. Wenn ich sie brauche, sag mir den **Dateipfad**.
- **Android geht ohne Apple**. Zieht sich Schritt 1, läuft Android schon — der Server-Teil ist derselbe.

1 Apple-Mitgliedschaft auf diesem Mac

PFLICHT · 10 MIN

Warum: Ohne bezahltes Team darf die App keine Push-Benachrichtigungen verwenden, und es gibt kein Distribution-Zertifikat — derselbe Grund, aus dem `flutter build ipa` scheitert.

1. Xcode öffnen.
2. Menü **Xcode** → **Settings...** (oder `⌘ + ,`).
3. Reiter **Accounts**.
4. Unten links auf **+** → **Apple ID** → **Continue**.
5. Mit der Apple ID anmelden, die zum **bezahlten** Team `HACJC6623Z` gehört.
6. Links das Konto anklicken. Rechts muss **MatchUp** (`HACJC6623Z`) stehen — Rolle *Account Holder*, *Admin* oder *App Manager*.
7. Rechts unten **Download Manual Profiles**.

```
security find-identity -v -p codesigning
```

Fertig, wenn dort eine Zeile mit *Apple Distribution* steht. Steht weiterhin nur *Apple Development*, bist du im falschen Team angemeldet oder die Mitgliedschaft ist abgelaufen — das ist der Stand von heute.

2 Push für die App-ID einschalten

PFLICHT · 3 MIN

Warum: Apple muss wissen, dass genau diese App Push benutzen darf. Ohne das Häkchen trägt das Provisioning-Profil kein `aps-environment`, und APNs weist jede Nachricht ab.

1. `developer.apple.com/account` öffnen, anmelden.
2. Oben rechts prüfen: Steht dort **MatchUp / HACJC6623Z**? Sonst umschalten — mit dem persönlichen Team gibt es die Option nicht.
3. Links **Certificates, Identifiers & Profiles** → **Identifiers**.
4. In der Liste `app.matchup.mobile` anklicken.
5. Häkchen bei **Push Notifications** setzen.
6. Oben rechts **Save**, Rückfrage mit **Confirm**.
7. **Danach in Xcode** noch einmal **Download Manual Profiles** — sonst baut Xcode weiter gegen das alte Profil ohne Push.

Fertig, wenn neben *Push Notifications* ein Häkchen steht und die Seite *Enabled* anzeigt.

3 Die Schlüsseldatei .p8

ERLEDIGT

Steht seit dem 24.09.2026: ~/keys/AuthKey_9JF35BFS9Q.p8 . Die Anleitung bleibt stehen, falls der Schlüssel je ersetzt werden muss.

Warum: Sie ist der eigentliche Schlüssel. Key ID und Team ID sind nur die Kennnummern dazu.

Apple gibt eine .p8 nur einmal heraus. Ist sie weg, legst du einen neuen Schlüssel an:

developer.apple.com/account → **Keys** → **+** → Name MatchUp Push → Häkchen **APNs** → **Continue** → **Register** → **Download**. Die neue Key ID notieren und bei Firebase eintragen.

Widerrufen nur, was wirklich verloren ist. Hier stand bis zum 04.10.2026 die Key ID des *aktiven* Schlüssels — sie war beim Nachziehen der neuen Nummer versehentlich mitersetzt worden. Wer dem folgte, widerrief den Schlüssel, den er gerade benutzt, und bekam danach `Invalid APNs credential`.

```
mkdir -p ~/keys && mv ~/Downloads/AuthKey_*.p8 ~/keys/
```

4 Firebase-Projekt

ERLEDIGT

Steht seit dem 21.09.2026: Projekt matchup-f9e83 , Spark-Tarif (kostenlos), Analytics aus.

Warum: Firebase Cloud Messaging verschickt an iOS **und** Android über einen Weg. Ohne Firebase bräuchte es zwei Versandwege.

5 Die beiden Apps in Firebase

ERLEDIGT

Beide Konfigdateien liegen im Projekt: android/app/google-services.json und ios/Runner/GoogleService-Info.plist .

Sie sind keine Geheimnisse und gehören ins Repo — sie liegen in jedem ausgelieferten Build, und ohne sie baut das Projekt auf einem zweiten Rechner nicht. In .gitignore stehen nur die echten Geheimnisse: *.p8 und *firebase-adminsdk*.json .

6 APNs-Schlüssel bei Firebase

HIER HÄNGT ES

Am 04.10.2026 gemessen: Der Schlüssel greift nicht. Ein Testpush an das registrierte iOS-Gerät scheitert mit HTTP 401 · Invalid APNs credential (THIRD_PARTY_AUTH_ERROR).

Was das heißt: FCM hat den Geräte-Token **angenommen** — sonst käme UNREGISTERED — und ist dann bei Apple abgewiesen worden. Der Fehler sitzt zwischen Firebase und Apple. **Die Datei auf dem Mac ist dabei in Ordnung:** gültiger privater Schlüssel, 256 Bit, 257 Bytes. **Nachzusehen ist zweierlei:** ob bei Firebase die Key ID 9JF35BFS9Q und die Team ID HACJC6623Z stehen (es gab einen Vorgänger 8887U4P9BH) — und ob der Schlüssel bei Apple noch aktiv ist und nicht auf *Revoked* steht.

Warum überhaupt: Firebase spricht in deinem Namen mit Apple. Dafür braucht es die .p8 samt Kennnummern.

1. Firebase-Konsole → Zahnrad → **Projekteinstellungen** → Reiter **Cloud Messaging**.
2. Abschnitt **Apple-App-Konfiguration**, bei `app.matchup.mobile`: **APNs-Authentifizierungsschlüssel** → **Hochladen**.
3. **Datei:** die .p8 aus `~/keys/`. **Key ID:** 9JF35BFS9Q . **Team ID:** HACJC6623Z .

7 Dienstkonto für den Versand

ERLEDIGT

Steht. Die Datei liegt als `~/keys/firebase-dienstkonto.json` und seit dem 24.09.2026 als Supabase-Secret FIREBASE_DIENSTKONTO — im Klartext ist sie nirgends aufgetaucht. Am 02.10.2026 nachgemessen: Sie holt einen OAuth-Token, FCM antwortet.

Vollzugriff-Schlüssel. Nicht weitergeben, nicht ins Repo.

Am 25.09.2026 veröffentlicht: `sleeperdach.github.io/datenschutz.html` zeigt Abschnitt 6 („Push-Benachrichtigungen“), die Quelle liegt als `docs/datenschutz.html` im Projekt. Beim Übernehmen wurde der `<!DOCTYPE html>` ergänzt — die Fassung begann direkt mit `<html>`, was Browser in den Quirks-Modus schickt; am Wortlaut wurde nichts geändert.

Offen bleiben die beiden Store-Angaben. Push erhebt genau **eine** zusätzliche Datenart — die Geräte-Kennung. Kein Tracking, keine Werbung.

APP STORE CONNECT → APP-DATENSCHUTZ	ANTWORT
Zusätzlicher Datentyp	Identifikatoren → Geräte-ID
Verwendungszweck	App-Funktionalität
Mit der Identität verknüpft?	Ja — der Token hängt am Konto
Für Tracking verwendet?	Nein

PLAY CONSOLE → DATENSICHERHEIT	ANTWORT
Zusätzlicher Datentyp	Geräte- oder andere IDs
Erhoben? / Geteilt?	Ja / Nein — Google ist Auftragsverarbeiter
Zweck	App-Funktionalität
Pflicht oder optional?	Optional — der Nutzer kann Push ablehnen
Verschlüsselt? / Löschbar?	Ja / Ja

Der Server läuft – nachgemessen

WAS	NACHGEMESSEN
Migration 0129	<code>push_geraete</code> , <code>push_einstellungen</code> , <code>push_auftraege</code> stehen
Secret	<code>FIREBASE_DIENSTKONT0</code> gesetzt; holt einen OAuth-Token (02.10.)
Function	ausgespielt mit <code>--no-verify-jwt</code> , antwortet <code>{"auftraege":0,"gesendet":0}</code>
Cron	<code>push-versand</code> (jede Minute), <code>push-tipp-erinnerungen</code> (alle 15 Min), <code>push-korb-aufräumen</code> (04:17) — alle drei aktiv
Auslöser	10 Trigger an Chats, Trades, Waivern, Ausfällen und Anfragen
Gateway	der Minutenlauf steht mit <code>200</code> in <code>net._http_response</code> — kein 401

Dass `gesendet` auf 0 steht, ist kein Fehler: Solange kein Gerät einen Token gemeldet hat, bleibt der Korb leer. Dafür braucht es Schritt 1 und 2 — und ein echtes iPhone.

Was Push auslösen soll

EREIGNIS	QUELLE
Du bist im Draft am Zug	<code>fantasy_leagues</code> (Zählerstand)
Trade-Angebot: neu, angenommen, abgelehnt	<code>fantasy_trades</code>
Waiver durchgegangen oder nicht	<code>fantasy_waiver_claims</code>
Direktnachricht, Liga- oder Tipprunden-Chat	<code>direct_messages</code> , <code>*_messages</code>
Aufgestellter Spieler fällt aus	<code>player_absences</code>
Tipps offen, kurz vor Deadline	<code>tip_rounds</code>
Beitritts- oder Freundschaftsanfrage	<code>*_join_requests</code> , <code>friendships</code>

MatchUp · Push-Benachrichtigungen · Stand 04.10.2026 · Quelle dieser Seite: `docs/push.html` , gebaut mit `tools/push_pdf.sh` · Inhaltlich gleich: `PUSH.md`